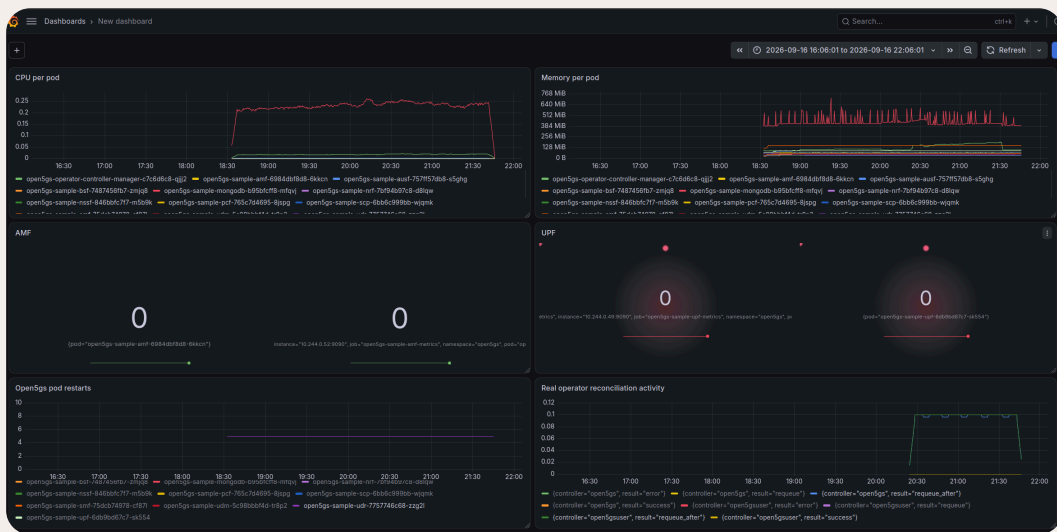


GITOPS-DRIVEN 5G CORE ON KUBERNETES

By Ahmed Tetteh | September 2026



Preparing the GitOps Development Environment

Project goals

In this project, I'm building and deploying an open-source 5G Core (Open5GS) on Kubernetes that mirrors the exact architecture KT Cloud uses in production.

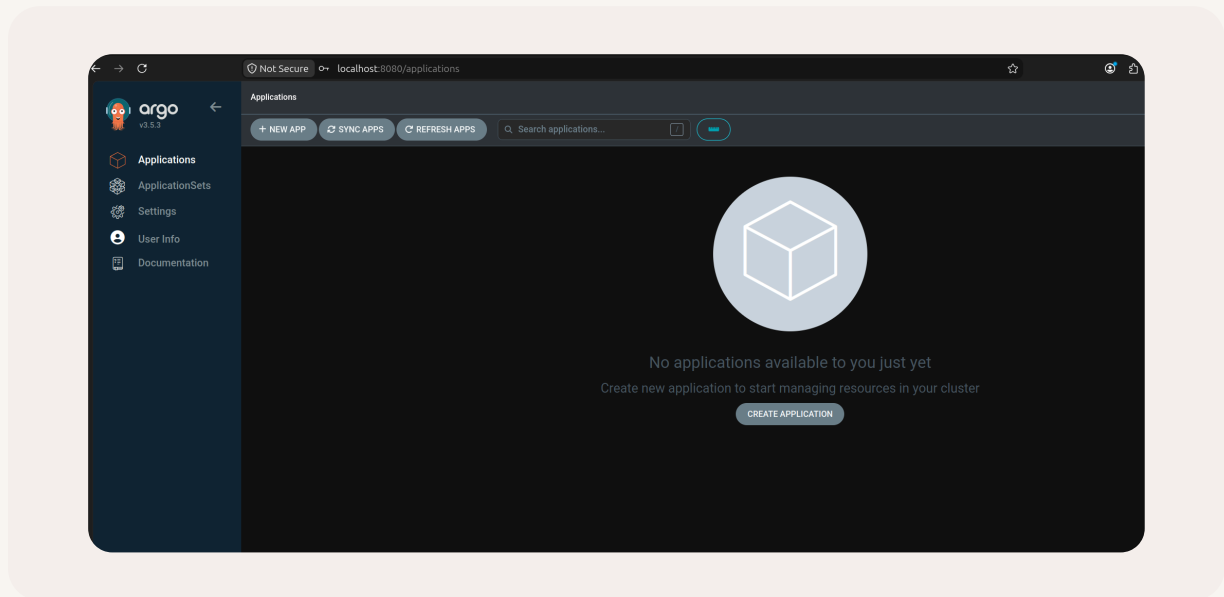
Verifying Docker and ArgoCD tooling

I verified ArgoCD CLI and Docker Desktop are installed correctly by running "argocd version --client" and "docker info".

Creating a Kubernetes Cluster for the 5G Core

Cluster setup objectives

In this step, I'm creating the 5G core cluster in Docker Desktop, so ArgoCd can manage that cluster for me.



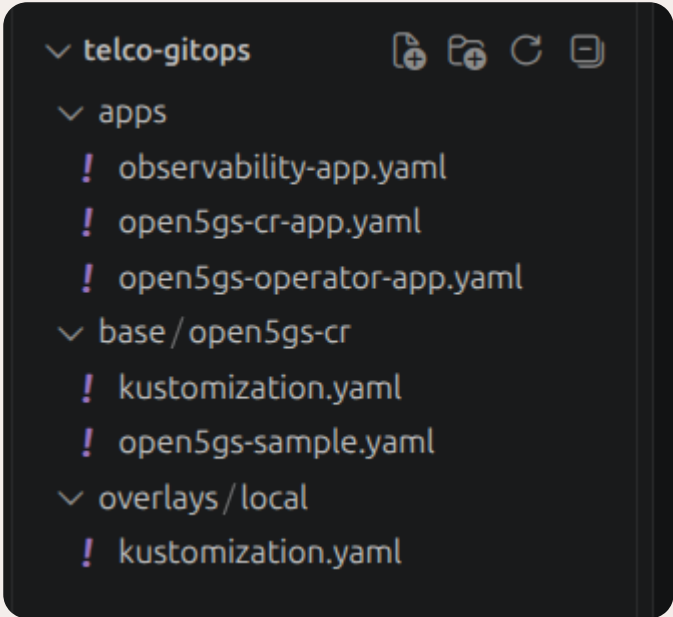
Kubernetes and Argo CD compatibility

My cluster is running version 1.36.4 because ArgoCD 3.5 officially supports that specific version of kubernetes.

Structuring a GitOps Repository

GitOps repository goals

In this step, I'm setting up the Git repository that ArgoCD will manage for sync my desired in Git to the cluster.



```
telco-gitops
├── apps
│   ├── ! observability-app.yaml
│   ├── ! open5gs-cr-app.yaml
│   └── ! open5gs-operator-app.yaml
├── base/open5gs-cr
│   ├── ! kustomization.yaml
│   └── ! open5gs-sample.yaml
└── overlays/local
    └── ! kustomization.yaml
```

The image shows a file explorer window with a dark theme. The root directory is 'telco-gitops'. It contains three main subdirectories: 'apps', 'base/open5gs-cr', and 'overlays/local'. Each of these subdirectories contains one or more files, each preceded by a red exclamation mark (!). The files are: 'observability-app.yaml', 'open5gs-cr-app.yaml', and 'open5gs-operator-app.yaml' in the 'apps' directory; 'kustomization.yaml' and 'open5gs-sample.yaml' in the 'base/open5gs-cr' directory; and 'kustomization.yaml' in the 'overlays/local' directory. The file explorer has standard icons for adding, deleting, refreshing, and opening a new window.

Apps and base directory responsibilities

The apps/ directory holds the ArgoCD application manifests, while the base/ directory holds the manifests for the resources that are created as the foundation per environment.

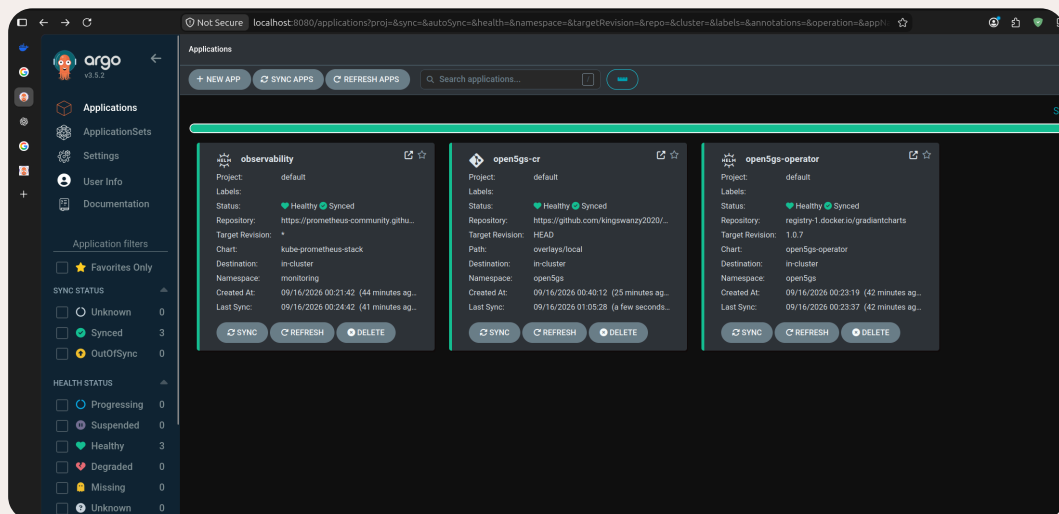
Deploying the 5G Core with ArgoCD

Deployment objectives

In this step, I'm deploying the 5G core via ArgoCD GitOps in the declarative way, instead of the imperative way.

Managing deployment order

I deployed the kub-prometheus observability stack first because the ServiceMonitor CRDs need to exist first for the open5Gs operator to create the ServiceMonitors for each network function.



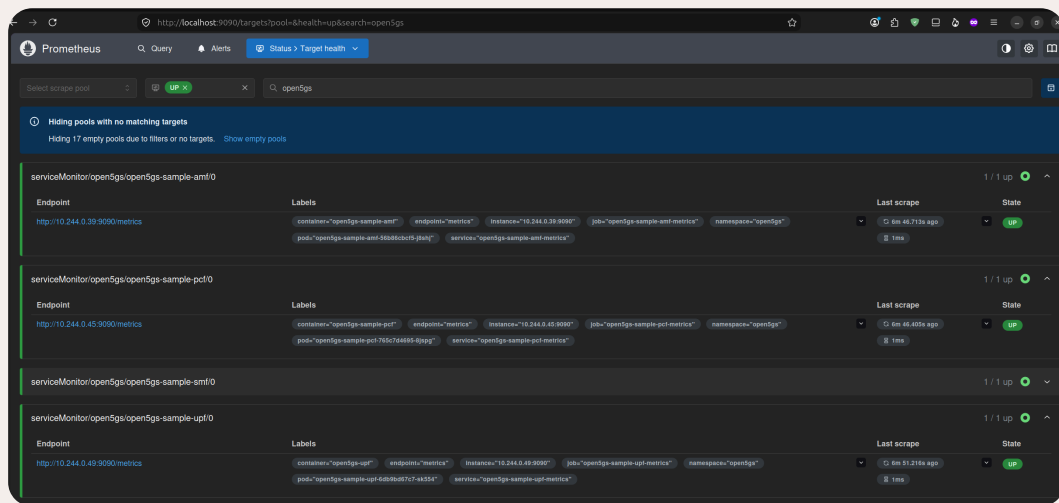
Verifying Open5GS network function pods

I can see 8 network function pods running, including a MongoDB pod for storing subsribing data.

Validating 5G Core Observability

Observability pipeline goals

In this step, I'm verifying that the observability pipeline is working end-to-end, such that the Open5GS ServiceMonitors are actively scraping metrics from the AMF, PCF, and UPF network functions.



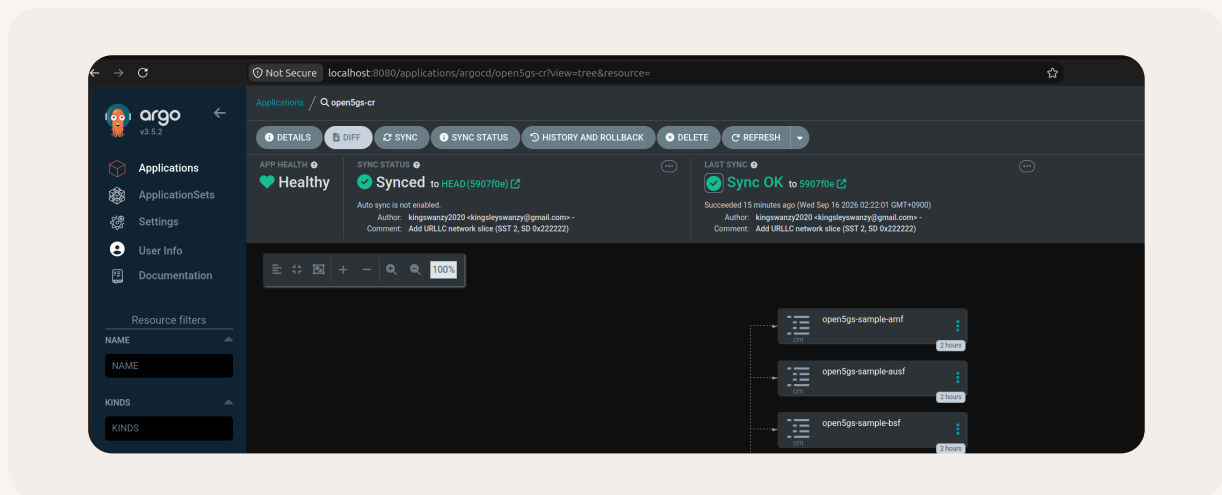
Open5GS metrics collected by Prometheus

The three network functions with metrics enabled are AMF (Access and Mobility Management Function), PCF(Policy Control Function) and UPF(User Plane Function).

Reconfiguring Network Slices Through GitOps

Slice reconfiguration objectives

In this step, I'm modifying network slices in my Git repository, so that I can verify that ArgoCD and the Open5GS Operator will handle the rollout automatically to match my desired and cluster state.



From Git commit to pod restart

I committed a change in the open5GS custom resource file by adding one more network slice to provide addition service functionality. ArgoCD detected change in my git repository, and initiated a sync, then the Operator reconciled my new desired state to the cluster by creating the second network slice and restarting the ones affected.

Extending Visibility with a Custom Grafana Dashboard

Using the Operator pattern to maintain subscriber consistency

In this project extension, the Operator detects drift by watching for changes in configuration files applied to custom resources its namespace, and corrects it by reconciling to the desired state within the resource manifest.

Reflecting on the GitOps 5G Core Build

Tools and concepts learned

The key tools I used include GitOps, VS Code. Key concepts I learnt include:

- Deploying a full 5G Standalone Core (AMF, SMF, UPF, NRF, UDM, AUSF, PCF) as Cloud-Native Network Functions on Kubernetes using the Operator pattern with automatic drift correction.
- Building a telco-grade observability pipeline with Prometheus and Grafana monitoring CNF metrics via ServiceMonitors.

Project duration and challenges

This project took approximately 3 hours to complete. The most challenging aspects were troubleshooting the ArgoCD OutOfSync status and designing a Git repository structure that closely mirrors the conventions used by production teams as leading telecommunications companies, such as KT Cloud, while keeping the entire setup within a single Git repository.

Next learning goals

I did this project today to learn how to Production teams in Telcos companies utilize the GitOps workflow to automate their services.